With this "fast" matrix multiplication, Strassen also obtained a surprising result for matrix inversion [1]. Suppose that the matrices

$$\begin{pmatrix} a_{00} & a_{01} \\ a_{10} & a_{11} \end{pmatrix} \quad \text{and} \quad \begin{pmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{pmatrix} \tag{2.11.5}$$

are inverses of each other. Then the c's can be obtained from the a's by the following operations (compare equations 2.7.11 and 2.7.25):

$$\begin{aligned} R_0 &= \text{Inverse}(a_{00}) \\ R_1 &= a_{10} \times R_0 \\ R_2 &= R_0 \times a_{01} \\ R_3 &= a_{10} \times R_2 \\ R_4 &= R_3 - a_{11} \\ R_5 &= \text{Inverse}(R_4) \\ c_{01} &= R_2 \times R_5 \\ c_{10} &= R_5 \times R_1 \\ R_6 &= R_2 \times c_{10} \\ c_{00} &= R_0 - R_6 \\ c_{11} &= -R_5 \end{aligned} \tag{2.11.6}$$

In (2.11.6) the "inverse" operator occurs just twice. It is to be interpreted as the reciprocal if the a's and c's are scalars, but as matrix inversion if the a's and c's are themselves submatrices. Imagine doing the inversion of a very large matrix, of order $N = 2^m$, recursively by partitions in half. At each step, halving the order *doubles* the number of inverse operations. But this means that there are only N divisions in all! So divisions don't dominate in the recursive use of (2.11.6). Equation (2.11.6) is dominated, in fact, by its 6 multiplications. Since these can be done by an $N^{\log_2 7}$ algorithm, so can the matrix inversion!

This is fun, but let's look at practicalities: If you estimate how large N has to be before the difference between exponent 3 and exponent $\log_2 7 = 2.807$ is substantial enough to outweigh the bookkeeping overhead, arising from the complicated nature of the recursive Strassen algorithm, you will find that LU decomposition is in no immediate danger of becoming obsolete. However, the fast matrix multiplication routine itself is beginning to appear in libraries like BLAS, where it is typically used for $N \gtrsim 100$.

Strassen's original result for matrix multiplication has been steadily improved. The fastest currently known algorithm [2] has an asymptotic order of $N^{2.376}$, but it is not likely to be practical to implement it.

If you like this kind of fun, then try these: (1) Can you multiply the complex numbers $(a+ib)$ and $(c+id)$ in only *three* real multiplications? [Answer: See §5.5.] (2) Can you evaluate a general fourth-degree polynomial in x for many different values of x with only *three* multiplications per evaluation? [Answer: See §5.1.]

CITED REFERENCES AND FURTHER READING:

Strassen, V. 1969, "Gaussian Elimination Is Not Optimal," *Numerische Mathematik*, vol. 13, pp. 354–356.[1]

Coppersmith, D., and Winograd, S. 1990, "Matrix Multiplications via Arithmetic Progressions," *Journal of Symbolic Computation*, vol. 9, pp. 251–280.[2]

Kronsjö, L. 1987, *Algorithms: Their Complexity and Efficiency*, 2nd ed. (New York: Wiley).

Winograd, S. 1971, "On the Multiplication of 2 by 2 Matrices," *Linear Algebra and Its Applications*, vol. 4, pp. 381–388.

Pan, V. Ya. 1980, "New Fast Algorithms for Matrix Operations," *SIAM Journal on Computing*, vol. 9, pp. 321–342.

Pan, V. 1984, *How to Multiply Matrices Faster*, Lecture Notes in Computer Science, vol. 179 (New York: Springer)

Pan, V. 1984, "How Can We Speed Up Matrix Multiplication?", *SIAM Review*, vol. 26, pp. 393–415.

CHAPTER 3

Interpolation and Extrapolation

3.0 Introduction

We sometimes know the value of a function $f(x)$ at a set of points $x_0, x_1, \ldots,$ x_{N-1} (say, with $x_0 < \ldots < x_{N-1}$), but we don't have an analytic expression for $f(x)$ that lets us calculate its value at an arbitrary point. For example, the $f(x_i)$'s might result from some physical measurement or from long numerical calculation that cannot be cast into a simple functional form. Often the x_i's are equally spaced, but not necessarily.

The task now is to estimate $f(x)$ for arbitrary x by, in some sense, drawing a smooth curve through (and perhaps beyond) the x_i. If the desired x is in between the largest and smallest of the x_i's, the problem is called *interpolation*; if x is outside that range, it is called *extrapolation*, which is considerably more hazardous (as many former investment analysts can attest).

Interpolation and extrapolation schemes must model the function, between or beyond the known points, by some plausible functional form. The form should be sufficiently general so as to be able to approximate large classes of functions that might arise in practice. By far most common among the functional forms used are polynomials (§3.2). Rational functions (quotients of polynomials) also turn out to be extremely useful (§3.4). Trigonometric functions, sines and cosines, give rise to *trigonometric interpolation* and related Fourier methods, which we defer to Chapters 12 and 13.

There is an extensive mathematical literature devoted to theorems about what sort of functions can be well approximated by which interpolating functions. These theorems are, alas, almost completely useless in day-to-day work: If we know enough about our function to apply a theorem of any power, we are usually not in the pitiful state of having to interpolate on a table of its values!

Interpolation is related to, but distinct from, *function approximation*. That task consists of finding an approximate (but easily computable) function to use in place of a more complicated one. In the case of interpolation, you are given the function f at points *not of your own choosing*. For the case of function approximation, you are allowed to compute the function f at *any* desired points for the purpose of developing

your approximation. We deal with function approximation in Chapter 5.

One can easily find pathological functions that make a mockery of any interpolation scheme. Consider, for example, the function

$$f(x) = 3x^2 + \frac{1}{\pi^4} \ln\left[(\pi - x)^2\right] + 1 \tag{3.0.1}$$

which is well-behaved everywhere except at $x = \pi$, very mildly singular at $x = \pi$, and otherwise takes on all positive and negative values. Any interpolation based on the values $x = 3.13, 3.14, 3.15, 3.16$, will assuredly get a very wrong answer for the value $x = 3.1416$, even though a graph plotting those five points looks really quite smooth! (Try it.)

Because pathologies can lurk anywhere, it is highly desirable that an interpolation and extrapolation routine should provide an estimate of its own error. Such an error estimate can never be foolproof, of course. We could have a function that, for reasons known only to its maker, takes off wildly and unexpectedly between two tabulated points. Interpolation always presumes some degree of smoothness for the function interpolated, but within this framework of presumption, deviations from smoothness can be detected.

Conceptually, the interpolation process has two stages: (1) Fit (once) an interpolating function to the data points provided. (2) Evaluate (as many times as you wish) that interpolating function at a target point x.

However, this two-stage method is usually not the best way to proceed in practice. Typically it is computationally less efficient, and more susceptible to roundoff error, than methods that construct a functional estimate $f(x)$ directly from the N tabulated values every time one is desired. Many practical schemes start at a nearby point $f(x_i)$, and then add a sequence of (hopefully) decreasing corrections, as information from other nearby $f(x_i)$'s is incorporated. The procedure typically takes $O(M^2)$ operations, where $M \ll N$ is the number of local points used. If everything is well behaved, the last correction will be the smallest, and it can be used as an informal (though not rigorous) bound on the error. In schemes like this, we might also say that there are two stages, but now they are: (1) Find the right starting position in the table (x_i or i). (2) Perform the interpolation using M nearby values (for example, centered on x_i).

In the case of polynomial interpolation, it sometimes does happen that the coefficients of the interpolating polynomial are of interest, even though their use in *evaluating* the interpolating function should be frowned on. We deal with this possibility in §3.5.

Local interpolation, using M nearest-neighbor points, gives interpolated values $f(x)$ that do not, in general, have continuous first or higher derivatives. That happens because, as x crosses the tabulated values x_i, the interpolation scheme switches which tabulated points are the "local" ones. (If such a switch is allowed to occur anywhere *else*, then there will be a discontinuity in the interpolated function itself at that point. Bad idea!)

In situations where continuity of derivatives is a concern, one must use the "stiffer" interpolation provided by a so-called *spline* function. A spline is a polynomial between each pair of table points, but one whose coefficients are determined "slightly" nonlocally. The nonlocality is designed to guarantee global smoothness in the interpolated function up to some order of derivative. Cubic splines (§3.3) are the

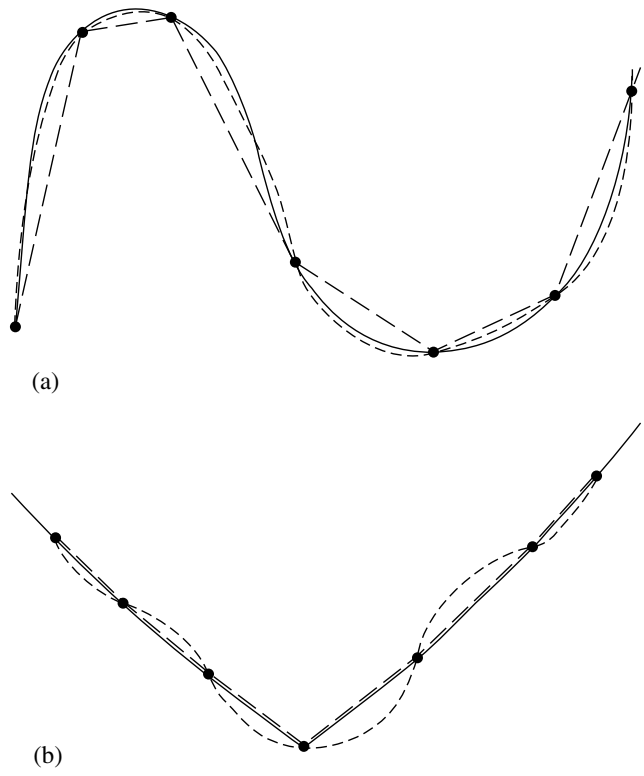

Figure 3.0.1. (a) A smooth function (solid line) is more accurately interpolated by a high-order polynomial (shown schematically as dotted line) than by a low-order polynomial (shown as a piecewise linear dashed line). (b) A function with sharp corners or rapidly changing higher derivatives is *less* accurately approximated by a high-order polynomial (dotted line), which is too "stiff," than by a low-order polynomial (dashed lines). Even some smooth functions, such as exponentials or rational functions, can be badly approximated by high-order polynomials.

most popular. They produce an interpolated function that is continuous through the second derivative. Splines tend to be stabler than polynomials, with less possibility of wild oscillation between the tabulated points.

The number M of points used in an interpolation scheme, minus 1, is called the *order* of the interpolation. Increasing the order does not necessarily increase the accuracy, especially in polynomial interpolation. If the added points are distant from the point of interest x, the resulting higher-order polynomial, with its additional constrained points, tends to oscillate wildly between the tabulated values. This oscillation may have no relation at all to the behavior of the "true" function (see Figure 3.0.1). Of course, adding points *close* to the desired point usually does help, but a finer mesh implies a larger table of values, which is not always available.

For polynomial interpolation, it turns out that the *worst* possible arrangement of the x_i's is for them to be equally spaced. Unfortunately, this is by far the most common way that tabulated data are gathered or presented. High-order polynomial interpolation on equally spaced data is *ill-conditioned*: small changes in the data can give large differences in the oscillations between the points. The disease is particularly bad if you are interpolating on values of an analytic function that has poles in

the complex plane lying inside a certain oval region whose major axis is the M-point interval. But even if you have a function with no nearby poles, roundoff error can, in effect, create nearby poles and cause big interpolation errors. In §5.8 we will see that these issues go away if you are allowed to choose an optimal set of x_i's. But when you are handed a table of function values, that option is not available.

As the order is increased, it is typical for interpolation error to decrease at first, but only up to a certain point. Larger orders result in the error exploding.

For the reasons mentioned, it is a good idea to be cautious about high-order interpolation. We can enthusiastically endorse polynomial interpolation with 3 or 4 points; we are perhaps tolerant of 5 or 6; but we rarely go higher than that unless there is quite rigorous monitoring of estimated errors. Most of the interpolation methods in this chapter are applied *piecewise* using only M points at a time, so that the order is a fixed value $M - 1$, no matter how large N is. As mentioned, *splines* (§3.3) are a special case where the function and various derivatives are required to be continuous from one interval to the next, but the order is nevertheless held fixed a a small value (usually 3).

In §3.4 we discuss *rational function interpolation*. In many, but not all, cases, rational function interpolation is more robust, allowing higher orders to give higher accuracy. The standard algorithm, however, allows poles on the real axis or nearby in the complex plane. (This is not necessarily bad: You may be trying to approximate a function with such poles.) A newer method, *barycentric rational interpolation* (§3.4.1) suppresses all nearby poles. This is the only method in this chapter for which we might actually encourage experimentation with high order (say, > 6). Barycentric rational interpolation competes very favorably with splines: its error is often smaller, and the resulting approximation is infinitely smooth (unlike splines).

The interpolation methods below are also methods for extrapolation. An important application, in Chapter 17, is their use in the integration of ordinary differential equations. There, considerable care *is* taken with the monitoring of errors. Otherwise, the dangers of extrapolation cannot be overemphasized: An interpolating function, which is perforce an extrapolating function, will typically go berserk when the argument x is outside the range of tabulated values by more (and often significantly less) than the typical spacing of tabulated points.

Interpolation can be done in more than one dimension, e.g., for a function $f(x, y, z)$. Multidimensional interpolation is often accomplished by a sequence of one-dimensional interpolations, but there are also other techniques applicable to scattered data. We discuss multidimensional methods in §3.6 – §3.8.

CITED REFERENCES AND FURTHER READING:

Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at `http://www.nr.com/aands`, §25.2.

Ueberhuber, C.W. 1997, *Numerical Computation: Methods, Software, and Analysis*, vol. 1 (Berlin: Springer), Chapter 9.

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), Chapter 2.

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 3.

Johnson, L.W., and Riess, R.D. 1982, *Numerical Analysis*, 2nd ed. (Reading, MA: Addison-Wesley), Chapter 5.

Ralston, A., and Rabinowitz, P. 1978, *A First Course in Numerical Analysis*, 2nd ed.; reprinted 2001 (New York: Dover), Chapter 3.

Isaacson, E., and Keller, H.B. 1966, *Analysis of Numerical Methods*; reprinted 1994 (New York: Dover), Chapter 6.

3.1 Preliminaries: Searching an Ordered Table

We want to define an interpolation object that knows everything about interpolation except one thing — how to actually interpolate! Then we can plug mathematically different interpolation methods into the object to get different objects sharing a common user interface. A key task common to all objects in this framework is finding your place in the table of x_i's, given some particular value x at which the function evaluation is desired. It is worth some effort to do this efficiently; otherwise you can easily spend more time searching the table than doing the actual interpolation.

Our highest-level object for one-dimensional interpolation is an abstract base class containing just one function intended to be called by the user: `interp(x)` returns the interpolated function value at x. The base class "promises," by declaring a virtual function `rawinterp(jlo,x)`, that every derived interpolation class will provide a method for local interpolation when given an appropriate local starting point in the table, an offset `jlo`. Interfacing between `interp` and `rawinterp` must thus be a method for calculating `jlo` from x, that is, for searching the table. In fact, we will use two such methods.

interp_1d.h

```
struct Base_interp
```
Abstract base class used by all interpolation routines in this chapter. Only the routine `interp` is called directly by the user.
```
{
    Int n, mm, jsav, cor, dj;
    const Doub *xx, *yy;
    Base_interp(VecDoub_I &x, const Doub *y, Int m)
```
Constructor: Set up for interpolating on a table of `x`'s and `y`'s of length `m`. Normally called by a derived class, not by the user.
```
        : n(x.size()), mm(m), jsav(0), cor(0), xx(&x[0]), yy(y) {
        dj = MIN(1,(int)pow((Doub)n,0.25));
    }

    Doub interp(Doub x) {
```
Given a value `x`, return an interpolated value, using data pointed to by `xx` and `yy`.
```
        Int jlo = cor ? hunt(x) : locate(x);
        return rawinterp(jlo,x);
    }

    Int locate(const Doub x);          See definitions below.
    Int hunt(const Doub x);

    Doub virtual rawinterp(Int jlo, Doub x) = 0;
```
Derived classes provide this as the actual interpolation method.
```
};
```

Formally, the problem is this: Given an array of abscissas x_j, $j = 0, \ldots, N-1$, with the abscissas either monotonically increasing or monotonically decreasing, and given an integer $M \leq N$, and a number x, find an integer $j_{\rm lo}$ such that x is centered